

AI & LLM Fundamentals

The foundation every AI Engineer needs before touching a single prompt — what AI and LLMs actually are, how they generate text, and the core parameters (tokens, context window, temperature, top-p, top-k) that control their behavior.

DIFFICULTY LEVEL

Beginner

ESTIMATED PRACTICE TIME

90-120 minutes

ESTIMATED READING TIME

55-70 minutes

PREREQUISITES

None — beginner friendly

LEARNING OUTCOMES

By the end of this module, you will be able to: explain what AI and LLMs are in plain language; describe how an LLM generates text token-by-token; explain tokens, tokenization, and context windows; correctly tune temperature, top-p, top-k, and max tokens for a task; recognize and reduce hallucinations; and tell reasoning models apart from standard chat models.

Table of Contents

1.	Introduction
2.	Before You Begin
3.	Main Lesson
3.1	What is AI?
3.2	What is an LLM?
3.3	How LLMs Generate Text
3.4	Tokens
3.5	Tokenization
3.6	Context Window
3.7	Temperature
3.8	Top-p
3.9	Top-k
3.10	Max Tokens
3.11	Hallucinations
3.12	Reasoning Models vs Chat Models
3.13	Model Capabilities & Limitations
4.	Visual Learning
5.	Real World Applications
6.	Practical Examples
7.	Code Examples
8.	Common Mistakes
9.	Best Practices
10.	Hands-on Activities
11.	Mini Project
12.	Review Questions
13.	Cheat Sheet
14.	Key Takeaways
15.	Additional Resources

1. Introduction

What is this topic?

This module is the ground floor of the entire "Prompt Engineering Mastery" bucket. Before you can write a good prompt, call an API, or build an agent, you need a working mental model of what an AI model actually is and how it decides what to say next. That's what this module builds.

Why is it important?

Every advanced topic later in this course — prompting techniques, structured outputs, RAG, agents — is really just "different ways of shaping what goes into an LLM, and different ways of controlling what comes out." Once you understand tokens, context windows, and sampling parameters, those later topics stop feeling like magic and start feeling like engineering decisions.



Why should AI Engineers learn it?

An AI Engineer who understands these fundamentals can debug weird model behavior instead of guessing, control cost (tokens = money), control output quality (temperature/top-p tuning), and explain to a teammate or a client *why* the model did what it did.

Where is it used in the real world?

Every product built on top of an LLM — chatbots, coding assistants, search copilots, customer support bots, study assistants — depends on someone understanding these fundamentals well enough to configure the model correctly for the task.

Why is it relevant today?

As of 2026, LLMs like Claude, GPT, and Gemini power a large and fast-growing share of new software features. It's not optional background knowledge anymore — it's the entry ticket.

Note

This module is deliberately non-mathematical. You do not need linear algebra or transformer internals to use LLMs well as an engineer — we focus on the mental models and controls you actually touch day to day.

2. Before You Begin

This module assumes zero prior AI/ML knowledge. Here are the only two things worth knowing before you start:

TERM

Software vs. AI model — the difference in how they're built.

SIMPLE EXPLANATION

Traditional software runs on rules a human programmer wrote explicitly ("if age > 18, allow access"). An AI model instead *learns* patterns from huge amounts of example data, and no single human wrote down the exact rule it ends up using.

EVERYDAY EXAMPLE

A traditional spam filter might use a rule: "if email contains 'FREE MONEY', mark as spam."
An AI spam filter instead looked at millions of emails labeled spam/not-spam and learned the pattern itself.

TERM

Model vs. product — what you're actually interacting with.

SIMPLE EXPLANATION

"Claude" or "GPT" refers to the underlying AI model. "Claude.ai" or "ChatGPT" is the product/app built around that model (chat interface, memory, file upload, etc). As an AI Engineer, you'll mostly work with the model directly, through an API.



Tip

If any term below feels unfamiliar, don't skip ahead — every technical term in this guide is defined the moment it's introduced.

3. Main Lesson

Each concept below follows the same structure: **Definition** → **Simple Explanation** → **Analogy** → **Everyday Example** → **Why It Matters** → **Common Mistakes** → **Best Practices** → **Summary**. Take them in order — later concepts build on earlier ones.

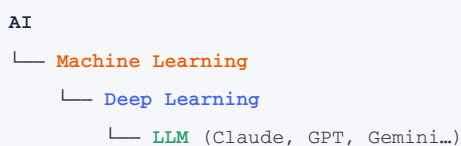
📦 3.1 What is AI?

DEFINITION

Artificial Intelligence (AI) is the broad field of building systems that perform tasks which normally require human intelligence — understanding language, recognizing images, making decisions, generating content.

SIMPLE EXPLANATION

AI is the umbrella term. Machine Learning (ML) is a subfield of AI where systems learn from data instead of being explicitly programmed. Deep Learning is a subfield of ML using layered neural networks. LLMs are a type of deep learning model trained on huge amounts of text.



💡 Think of AI as "vehicles," ML as "cars," Deep Learning as "electric cars," and LLMs as "a specific electric car model." Each is a narrower category inside the last.

EVERYDAY EXAMPLE

Google Maps predicting traffic (ML), your phone's face unlock (deep learning/computer vision), and Claude answering your question (LLM) are all "AI" — but very different techniques under the hood.

WHY IT MATTERS

People use "AI" as a catch-all, which causes confusion. Knowing where LLMs sit inside the bigger AI picture stops you from assuming an LLM works like every other kind of AI.

COMMON MISTAKES

Assuming "AI" always means an LLM/chatbot. Assuming all AI systems "think" the way humans do.

BEST PRACTICES

When someone says "AI," always ask "which kind?" — a recommendation engine, a computer vision model, and an LLM behave completely differently.

SUMMARY

AI is the umbrella field; LLMs are one specific, text-focused branch of it, and the rest of this course lives inside that branch.

🤖 3.2 What is an LLM?

DEFINITION

A Large Language Model (LLM) is a deep learning model trained on massive amounts of text to predict, given some input text, what text is likely to come next.

SIMPLE EXPLANATION

An LLM doesn't "know facts" the way a database does. It has learned statistical patterns of language well enough that predicting "the next likely word" over and over produces coherent, useful answers.

💡 *Imagine a friend who has read an enormous fraction of the internet and, without looking anything up, can continue any sentence you start in a way that "sounds right" based on everything they've read.*

EVERYDAY EXAMPLE

Claude, GPT-5, Gemini, and Llama are all LLMs. When you type a question, the model is really just predicting the most statistically likely reply, one small chunk of text at a time.

WHY IT MATTERS

This "next-token prediction" framing explains almost every LLM behavior you'll encounter later — including why models can sound extremely confident while being wrong (Section 3.11, Hallucinations).

COMMON MISTAKES

Treating an LLM's confident tone as a signal of correctness. Assuming it "looked something up" when it answered a factual question.

BEST PRACTICES

Treat LLM output as a highly capable first draft, not a verified fact-check — especially for anything numeric, legal, medical, or date-sensitive.

SUMMARY

An LLM is a next-token predictor trained on huge amounts of text — powerful because language patterns encode a surprising amount of real-world knowledge and reasoning ability.

⚙️ 3.3 How LLMs Generate Text

DEFINITION

Text generation is the repeated process of: look at everything so far → predict a probability distribution over possible "next tokens" → pick one → repeat.

SIMPLE EXPLANATION

The model never writes a whole sentence in one shot. It generates one token, appends it to the input, and generates the next token based on the new, longer input — over and over until it decides to stop.

"Rain"



"Rain falls"



"Rain falls softly"



...repeat...

💡 *Like predictive text on your phone keyboard, but dramatically more sophisticated — instead of guessing from your last 2-3 words, it considers the entire conversation so far.*

EVERYDAY EXAMPLE

Ask Claude "Write a haiku about rain" — internally, it generates the response one token at a time, each step conditioned on all previous tokens.

WHY IT MATTERS

This explains why streaming responses (text appearing word-by-word) is the model's natural mode, not a UI trick.

COMMON MISTAKES

Assuming the model "plans" the whole answer in advance. It doesn't — structure emerges from training patterns, not explicit forward planning.

BEST PRACTICES

If you need well-structured output, explicitly ask for structure ("first give an outline, then expand") rather than assuming the model will self-organize.

SUMMARY

Generation = repeated next-token prediction. Every other concept in this module is really a control knob on this one loop.

🔧 3.4 Tokens

DEFINITION

A token is the basic unit of text an LLM reads and writes — usually a chunk smaller than a word, roughly $\frac{3}{4}$ of a word on average in English.

SIMPLE EXPLANATION

Models don't see raw letters or full words. They see tokens: "unbelievable" might become "un" + "believ" + "able". Common short words are often a single token; rare or long words get split into pieces.

💡 *Think of tokens like LEGO bricks of language — some bricks are whole words, some are just word-fragments, and the model builds sentences by predicting one brick at a time.*

EVERYDAY EXAMPLE

The word "strawberry" might split into tokens like "straw" + "berry". This is exactly why models sometimes struggle with letter-counting tasks — they're reading token chunks, not individual letters.

WHY IT MATTERS

API pricing is per token (input + output). Context window limits are measured in tokens, not words or characters.

COMMON MISTAKES

Assuming "1 token $\approx$ 1 word" precisely. Forgetting non-English languages often use more tokens per word than English.

BEST PRACTICES

For cost/length estimates, use a tokenizer tool rather than guessing from word count.

SUMMARY

Tokens are the model's actual "reading unit." Words are a human convenience; tokens are what the model — and your API bill — actually deals with.

✂️ 3.5 Tokenization

DEFINITION

Tokenization is the process of converting raw text into a sequence of tokens (and back again) using a fixed vocabulary the model was trained with.

SIMPLE EXPLANATION

Before any text reaches the model, a tokenizer chops it into pieces from its known vocabulary (often 100,000+ possible tokens) and converts each piece into a number.

💡 *Like translating a sentence into a fixed set of numbered stamps from a stamp collection — the "sentence" the model sees is really just a sequence of stamp numbers.*

EVERYDAY EXAMPLE

Different model families use different tokenizers — one reason the same text can cost a different number of tokens on Claude vs. GPT vs. Gemini.

WHY IT MATTERS

Tokenization quirks explain many odd model behaviors: struggling with reversing words, counting letters, or careful arithmetic on large numbers.

COMMON MISTAKES

Assuming tokenization is identical across all model providers — it isn't.

BEST PRACTICES

For character-level precision, ask the model to work step-by-step (e.g., spell a word with spaces) or use a code tool instead of a single-shot answer.

SUMMARY

Tokenization is the text-to-numbers translation step every prompt goes through — invisible to you, but the root cause of several classic LLM quirks.

3.6 Context Window

DEFINITION

The context window is the maximum number of tokens (input + output combined) a model can "see" and reason over in a single request.

SIMPLE EXPLANATION

Everything the model can consider — system prompt, conversation history, uploaded documents, and its own reply — must fit inside this fixed token budget.

```
[ System Prompt ][ Conversation History ][ New Message ][ Model's Reply ]
|<----- total must fit inside context window ----->|
As the conversation grows, the OLDEST tokens fall off first.
```

💡 Think of it as a desk of fixed size. You can pile documents on it, but once the desk is full, older papers get pushed off the edge to make room for new ones.

EVERYDAY EXAMPLE

A model with a 200,000-token context window can "hold" roughly a 500-page book's worth of text before it starts losing access to the earliest parts.

WHY IT MATTERS

This is why very long chats start to "forget" early details. It's also why RAG (Module 07) exists.

COMMON MISTAKES

Assuming a bigger context window means the model reasons equally well across all of it — models often attend more to the beginning and end than the middle ("lost in the middle").

BEST PRACTICES

Put the most important instructions near the start or end of a long prompt. Trim irrelevant history instead of relying purely on window size.

SUMMARY

The context window is the model's total working memory per request — a hard limit that shapes prompt, conversation, and RAG design.

🔥 3.7 Temperature

DEFINITION

Temperature is a parameter (typically 0 to 1, sometimes up to 2) that controls how "random" or "safe" the model's token choices are.

SIMPLE EXPLANATION

Low temperature makes the model almost always pick the highest-probability token (focused, predictable). High temperature flattens the probabilities, letting it pick less-likely tokens more often (varied, sometimes weirder).

💡 *Temperature is like a "risk dial" for word choice — low temperature is ordering your usual dish every time; high temperature is letting the chef surprise you.*

EVERYDAY EXAMPLE

temperature=0 for a customer-support bot needing consistent, factual answers.

temperature=0.9 for brainstorming creative story ideas.

WHY IT MATTERS

Wrong temperature is one of the most common causes of "weird/inconsistent answers" (too high) or "robotic, repetitive" answers (too low).

COMMON MISTAKES

Leaving temperature high for tasks needing consistency (data extraction, classification, code generation).

BEST PRACTICES

Low temperature (0–0.3) for factual/structured tasks; moderate (0.5–0.8) for general writing; higher (0.8–1) only for open-ended brainstorming.

SUMMARY

Temperature controls the randomness of token selection — the single biggest lever for "predictable" vs. "creative" output.

🔗 3.8 Top-p

DEFINITION

Top-p (nucleus sampling) restricts the model to choosing only from the smallest set of tokens whose cumulative probability adds up to at least p (e.g., 0.9 = top 90%).

SIMPLE EXPLANATION

Top-p first throws away the long tail of very unlikely tokens, then samples from what's left — staying within the most plausible options without fixing an exact count.

💡 *Think of a multiple-choice exam where the model shows you just enough options to cover 90% of the "reasonable answer space" — then picks from there.*

EVERYDAY EXAMPLE

top_p=0.9 means: rank all possible next tokens by probability, keep adding until combined probability reaches 90%, discard the rest, then sample.

WHY IT MATTERS

Top-p adapts to how confident the model is at each step, often producing more natural results than temperature alone.

COMMON MISTAKES

Tuning both temperature and top-p aggressively at once — this compounds unpredictably.

BEST PRACTICES

Adjust one of temperature or top-p, not both. A common safe default is temperature-only tuning with top-p left at default.

SUMMARY

Top-p limits sampling to the most probable "chunk" of options, adapting dynamically instead of using a fixed cutoff.

🔗 3.9 Top-k

DEFINITION

Top-k restricts the model to sampling only from the k most probable next tokens, regardless of their combined probability mass.

SIMPLE EXPLANATION

Where top-p uses a probability threshold, top-k uses a fixed count. `top_k=40` means only the 40 most likely next tokens are eligible.

💡 If top-p is "keep options until we cover 90%," top-k is "just give me exactly the top 40 options, always."

EVERYDAY EXAMPLE

A low `top_k` makes output more focused and repetitive; a high `top_k` allows more variety, similar in effect to raising temperature.

WHY IT MATTERS

Not every API exposes top-k, but it matters for open-source model tooling (Llama, Mistral) and research papers.

COMMON MISTAKES

Assuming top-k is available and behaves identically across every provider — support varies.

BEST PRACTICES

Check your provider's docs before relying on top-k; temperature and top-p alone are usually enough control.

SUMMARY

Top-k caps the candidate pool to a fixed number of top tokens — a count-based cousin of top-p's percentage-based cutoff.

🔪 3.10 Max Tokens

DEFINITION

Max tokens is a request parameter that caps how many tokens the model is allowed to generate in its response.

SIMPLE EXPLANATION

It's a hard stop — once output reaches this many tokens, generation is cut off immediately, even mid-sentence.

💡 Like a word-count limit enforced by physically ripping the page out the moment you hit the limit — not by the writer choosing to stop.

EVERYDAY EXAMPLE

Setting `max_tokens=100` on a request that needs 400 tokens will produce a response that abruptly stops partway through.

WHY IT MATTERS

Max tokens controls cost and latency — and a too-low value is one of the most common causes of "cut off mid-answer" bugs.

COMMON MISTAKES

Setting `max_tokens` too low and blaming the model. Confusing `max_tokens` with the context window.

BEST PRACTICES

Estimate realistic answer length and set `max_tokens` with headroom; for JSON output, leave room for closing brackets.

SUMMARY

Max tokens is your output length ceiling — set too low, it silently truncates answers.

🧠 3.11 Hallucinations

DEFINITION

A hallucination is when an LLM generates text that sounds fluent and confident but is factually incorrect or entirely made up.

SIMPLE EXPLANATION

Because the model predicts "statistically plausible next tokens," not database lookups, it can produce a confident-sounding but false citation, date, or statistic.

💡 *Like a well-read student answering an exam question from memory under time pressure — if they don't know the answer, they'll often write something plausible rather than leave it blank.*

EVERYDAY EXAMPLE

Asking an LLM to cite a research paper and receiving a title, author, and journal that all sound legitimate — but the paper doesn't exist.

WHY IT MATTERS

Hallucinations are the single biggest trust/safety issue in real LLM products.

COMMON MISTAKES

Trusting LLM output for facts or citations without verification. Assuming confident tone means correct answer.

BEST PRACTICES

Ground factual tasks with real documents (RAG) or tool calls; instruct the model to say "I don't know" rather than guess; verify high-stakes claims independently.

SUMMARY

Hallucinations are confident-sounding fabrications — manage them with grounding and verification, not blind trust.

🧠 3.12 Reasoning Models vs Chat Models

DEFINITION

Chat models generate a direct response quickly. Reasoning models generate an internal chain of intermediate reasoning steps before producing a final answer, trading speed for accuracy on hard problems.

SIMPLE EXPLANATION

A standard chat model answers in one fast pass. A reasoning model "thinks" first — working through a problem step-by-step — better for math, multi-step logic, and complex coding, but slower and pricier.

💡 *A chat model is answering a pop quiz instantly off the top of your head. A reasoning model gets scratch paper and time to work the problem out first.*

EVERYDAY EXAMPLE

Extended thinking modes in Claude are designed for problems where a quick guess is likely wrong.

WHY IT MATTERS

Picking the wrong mode wastes either money/time or accuracy.

COMMON MISTAKES

Always defaulting to the fastest model even for genuinely hard problems, or the opposite — expensive reasoning for simple lookups.

BEST PRACTICES

Match the mode to the task: simple answers → fast chat; multi-step logic → reasoning mode.

SUMMARY

Reasoning models spend more compute "thinking" before answering — better for hard problems, overkill for simple ones.

 3.13 Model Capabilities & Limitations

DEFINITION

The realistic boundary of what an LLM can reliably do well versus what it tends to struggle with, regardless of prompting.

SIMPLE EXPLANATION

LLMs are strong at language tasks, summarization, drafting, code generation. They are weaker at precise arithmetic, real-time information, and perfect factual recall without extra tooling.

Strong At	Weak At (without extra tooling)
Drafting & editing text	Precise large-number arithmetic
Summarizing & explaining	Real-time / current events
Code generation & debugging help	Perfectly citing exact sources
Pattern-based reasoning	Remembering past separate sessions
Following structured instructions	Guaranteed 100% factual accuracy

EVERYDAY EXAMPLE

An LLM will confidently summarize a 20-page PDF, but might get a 12-digit multiplication problem wrong unless given a calculator tool.

WHY IT MATTERS

Knowing this table before designing a feature tells you where you need extra scaffolding: tools for math/live data, RAG for grounded facts.

COMMON MISTAKES

Assuming a general-purpose LLM is a calculator, search engine, or database.

BEST PRACTICES

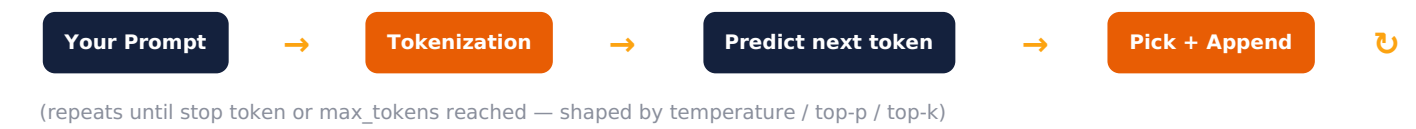
Pair the LLM with the right tool for what it's weak at (Module 06, Function Calling / Tool Use).

SUMMARY

LLMs are extraordinary language and reasoning engines, not omniscient databases or calculators.

4. Visual Learning

🔄The Generation Loop



📊Sampling Parameters at a Glance

Parameter	What it limits	Low value effect	High value effect
Temperature	How random token choice is	Focused, predictable, repetitive	Varied, creative, riskier
Top-p	Probability mass considered	Only near-certain tokens allowed	Wider pool of options allowed
Top-k	Fixed count of candidates	Very narrow candidate pool	Broad candidate pool
Max tokens	Output length	Short, possibly truncated	Long, costs more

🏠Context Window as a Desk

[System Prompt] [Conversation History] [New User Message] [Model's Reply]

|<————— total must fit inside context window —————>|

As the conversation grows, the OLDEST tokens are the first to fall off the desk.

📍Where LLMs Fit Inside AI

Artificial Intelligence

- Machine Learning (learns from data)
 - Deep Learning (neural networks)
 - Large Language Models (Claude, GPT, Gemini)

5. Real World Applications

Product	How these fundamentals show up
Claude / ChatGPT / Gemini (chat apps)	Context window limits determine how much chat history is "remembered"; temperature is tuned low for factual replies, higher for creative writing modes.
GitHub Copilot / Cursor / Claude Code	Low temperature for code completion; large context windows needed to "see" whole files/repos.
AI customer support bots	Low temperature + tight max_tokens + hallucination-mitigation (RAG grounding) for accurate, on-brand answers.
AI Agents (Module 08)	Reasoning-model modes used for multi-step planning; regular chat mode used for simple sub-tasks to control cost.

6. Practical Examples

Beginner: Spotting a hallucination

Ask any LLM: "Give me the exact page number where Einstein first mentions relativity in a specific obscure book." Notice it will likely give a confident-sounding page number — often fabricated.

Beginner: Feeling the context window

Have a very long conversation (50+ messages), then ask the model to recall something from your very first message. Past a certain length, it starts to lose track.

Intermediate: Temperature comparison

Send the identical prompt three times at `temperature=0`, and three times at `temperature=1`. Notice `temperature=0` responses are nearly identical each time; `temperature=1` responses vary noticeably.

Advanced: Token-aware cost estimation

Estimate: $\text{average input tokens} \times \text{requests per day} \times \text{price per token} = \text{daily cost}$.
Compare a chat feature with a 4,000-token average history versus a stateless single-question feature.

7. Code Examples

🐍Python — Basic Claude API call

```
# pip install anthropic
import anthropic

client = anthropic.Anthropic(api_key="YOUR_API_KEY")

response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=300,          # caps output length (Section 3.10)
    temperature=0.2,         # low = focused, factual answers (Section 3.7)
    top_p=0.9,               # nucleus sampling (Section 3.8)
    messages=[
        {"role": "user", "content": "Explain tokenization in two sentences."}
    ]
)

# response.content is a list of content blocks; extract the text
print(response.content[0].text)
```

📦JavaScript — Same call, Node.js

```
// npm install @anthropic-ai/sdk
import Anthropic from "@anthropic-ai/sdk";

const client = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY });

const response = await client.messages.create({
  model: "claude-sonnet-4-6",
  max_tokens: 300,          // output length cap
  temperature: 0.2,         // low randomness for a factual task
  messages: [
    { role: "user", content: "Explain tokenization in two sentences." }
  ],
});

console.log(response.content[0].text);
```

🐍Python — Rough token counting

```
# pip install anthropic
import anthropic

client = anthropic.Anthropic(api_key="YOUR_API_KEY")

count = client.messages.count_tokens(
    model="claude-sonnet-4-6",
    messages=[{"role": "user", "content": "How many tokens is this sentence?"}],
)

# Lets you estimate cost/context usage BEFORE spending a real API call
print(count.input_tokens)
```



Tip

Always estimate token usage for long prompts before running them in production loops — the easiest way to prevent silent cost overruns.

8. Common Mistakes

- **Trusting confident-sounding output as fact.** Fluency is not the same as accuracy.
- **Ignoring token costs until the bill arrives.** Long conversation histories silently multiply cost per request.
- **Using default temperature for every task.** A single global setting is rarely right for both precise extraction and wild brainstorming.
- **Confusing max_tokens with the context window.** They're different limits.
- **Assuming all providers tokenize identically.** Token counts (and cost) differ between Claude, GPT, and Gemini.
- **Using a heavy reasoning model for trivial tasks,** wasting time and money.

9. Best Practices

- **Match parameters to the task.** Low temperature + low top-p for factual work; higher temperature for brainstorming.
- **Always set max_tokens with realistic headroom**, especially for structured/JSON output.
- **Ground factual tasks with real data** (RAG, tool use) instead of relying on memorized knowledge.
- **Front-load or end-load critical instructions** in long contexts.
- **Estimate token cost before scaling** any AI feature to production traffic.
- **Pick chat vs. reasoning mode deliberately** based on task difficulty.

10. Hands-on Activities

Do these using any LLM chat interface or API playground you have access to. Answers are not provided — verify your own understanding by testing and observing the model's behavior.

Easy 5 exercises

1. In your own words (2–3 sentences), explain the difference between AI, Machine Learning, and an LLM.
2. Ask an LLM to spell out a word letter by letter with a dash between each letter. Notice how this sidesteps tokenization quirks.
3. Ask the same factual question to an LLM twice at temperature 0. Are the answers identical or different? Why?
4. Find the advertised context window size (in tokens) for three current models and write them down for comparison.
5. Ask an LLM a somewhat obscure question you already know the answer to. Check its answer for a hallucination.

Intermediate 3 exercises

1. Run the identical prompt at temperature 0.1 and temperature 1.0, three times each. Compare consistency.
2. Set `max_tokens` deliberately too low for a moderately long answer and observe the truncation.
3. Write a paragraph, then ask an LLM to count how many words it contains. Compare to the real count.

Challenge 2 exercises

1. Design (on paper) a system prompt + parameter config for two products: a legal-document summarizer and a children's bedtime-story generator. Justify each choice.
2. Write a short comparison (5–8 sentences) of how "reasoning mode" changes output on a genuinely hard logic puzzle, compared to standard chat mode.

11. Mini Project

Goal

Build a tiny command-line "Parameter Playground" script that sends the same prompt to Claude with different temperature values and compares outputs side by side.

Requirements

- An Anthropic API key
- Python or JavaScript with the Anthropic SDK installed
- A single hardcoded prompt of your choice

Step-by-Step Instructions

1. Set up your API key as an environment variable.
2. Write a loop that calls the Messages API three times with the same prompt, using `temperature = 0, 0.5, 1.0` (keep `max_tokens` fixed, e.g. 150).
3. Print each response clearly labeled with its temperature value.
4. Add a token-count printout for each response using the method from Section 7.
5. Run it 2–3 times: does temperature 0 stay identical across runs? Does temperature 1 vary?

Expected Output

```
=== Temperature 0 ===
[response text]
Tokens used: 42

=== Temperature 0.5 ===
[response text]
Tokens used: 47

=== Temperature 1.0 ===
[response text]
Tokens used: 51
```

Possible Improvements

- Add `top_p` as a second variable to compare against temperature.
- Log results to a CSV file for a small personal dataset of "parameter → output style" examples.
- Wrap it in a simple web UI — a great mini portfolio piece.

12. Review Questions

Multiple Choice (10)

1. What is the basic unit of text an LLM actually processes?

- A) Words B) Sentences C) Tokens D) Characters

2. What does the context window measure?

- A) Screen size B) Max tokens the model can consider per request C) Training data size D) Conversations stored

3. A temperature of 0 will generally produce:

- A) Maximum randomness B) The most focused, deterministic-leaning output C) No output at all D) Longer responses

4. Top-p sampling selects tokens based on:

- A) A fixed count of candidates B) Alphabetical order C) Cumulative probability mass D) Token length

5. Top-k sampling selects tokens based on:

- A) A fixed number of top candidates B) Cumulative probability C) Random shuffling D) Sentence position

6. What happens when a response hits the max_tokens limit?

- A) The model rephrases B) Generation stops immediately, even mid-sentence C) The request is rejected D) Context resets

7. A hallucination in LLM output refers to:

- A) A rendering bug B) Confident but false/made-up content C) A crashed request D) Excessive token usage

8. Reasoning models differ from chat models mainly by:

- A) Using more parameters always B) Spending more compute on intermediate steps before answering C) Never making mistakes D) Skipping tokenization

9. Which of these is typically weak for LLMs, without extra tools?

- A) Summarizing text B) Drafting emails C) Precise large-number arithmetic D) Explaining concepts

10. LLMs are a subfield of:

- A) Robotics B) Deep Learning, itself part of ML, itself part of AI C) Computer vision D) Databases

Identification (5)

1. _____ is the process of converting raw text into the model's numeric vocabulary units.

2. _____ is the parameter that caps how many tokens a response is allowed to generate.

3. _____ is the maximum combined input+output token capacity of a single request.

4. _____ keeps only the smallest set of tokens whose combined probability reaches a threshold p.

5. _____ describes confidently generated text that is factually false or fabricated.

True or False (5)

1. An LLM looks up facts in a live database when answering questions.

2. Temperature 0 generally produces more consistent output across repeated runs than temperature 1.

3. Max tokens and context window mean exactly the same thing.

4. All model providers use an identical tokenizer, so token counts are always the same across Claude, GPT, and Gemini.

5. Reasoning models are generally slower and more expensive per answer than standard chat models.

Situational (5)

1. You're building a legal-document summarizer needing highly consistent output. What temperature range would you choose, and why?

2. Your chatbot's answers keep getting cut off mid-sentence. Which parameter

should you check first?

3. A user asks your app to cite a statistic, and it gives a confident but wrong number. What phenomenon is this, and one mitigation?

4. Your app processes a 300-page PDF plus long chat history and the model starts "forgetting" earlier instructions. What concept explains this, and one fix?

5. You need reliable multi-step logic puzzle solving, but it's using a fast chat model and getting many wrong. What would you try next?

Answer Key

Multiple Choice: 1-C, 2-B, 3-B, 4-C, 5-A, 6-B, 7-B, 8-B, 9-C, 10-B

Identification: 1-Tokenization, 2-Max tokens, 3-Context window, 4-Top-p (nucleus sampling), 5-Hallucination

True or False: 1-False, 2-True, 3-False, 4-False, 5-True

Situational (sample reasoning):

1. Low temperature (0-0.3) — consistency and accuracy matter more than creative variation for legal text.
2. max_tokens — likely set too low for the expected response length.
3. Hallucination — mitigate with RAG grounding, tool-based fact lookup, or explicit "say you don't know" instructions.
4. Context window overflow — mitigate with summarization/trimming of older history, or RAG retrieving only relevant chunks.
5. Switch to a reasoning-mode model, since it involves genuine multi-step logic rather than simple recall.

13. Cheat Sheet

🔑 Core Terms

Token — smallest text unit the model reads/writes.

Tokenization — text → token conversion.

Context window — total input+output token budget.

Hallucination — confident but false output.

🔧 Sampling Controls

Temperature — randomness dial (low = focused, high = varied).

Top-p — probability-mass cutoff.

Top-k — fixed-count cutoff.

Max tokens — output length cap.

⚡ Quick Defaults

Factual/structured → temp 0-0.3

General writing → temp 0.5-0.8

Brainstorming → temp 0.8-1.0

Always verify high-stakes facts.

🎯 Model Choice

Simple / fast lookups → chat mode.

Multi-step logic, hard math → reasoning mode.

Match the tool to the task's real difficulty.

14. Key Takeaways

- AI is the broad field; LLMs are a specific text-focused branch built on deep learning.
- LLMs generate text by repeatedly predicting the next most likely token, not by "looking up" answers.
- Tokens (not words) are the real unit of cost, length, and model perception — and tokenization varies by provider.
- The context window is a hard cap on total input + output tokens per request; older content falls out first.
- Temperature, top-p, and top-k are the three main knobs for controlling randomness/creativity in output.
- Max tokens caps response length and directly affects cost and truncation risk.
- Hallucinations are confident fabrications — mitigate with grounding (RAG/tools), not blind trust.
- Reasoning models trade speed/cost for better accuracy on genuinely hard, multi-step problems.
- LLMs are excellent at language and pattern-based reasoning, but weak at precise math, live facts, and long-term memory without extra tooling.

15. Additional Resources

Official Documentation

- Anthropic — Claude API docs: docs.claude.com
- OpenAI — API documentation: platform.openai.com/docs
- Google — Gemini API documentation: ai.google.dev

Research Papers (foundational)

- "Attention Is All You Need" (Vaswani et al., 2017) — the transformer architecture underlying modern LLMs.
- "Language Models are Few-Shot Learners" (Brown et al., 2020) — the GPT-3 paper, on emergent capabilities from scale.

Tools

- Anthropic Console — token counting and prompt testing playground.
- Tiktoken (OpenAI's open-source tokenizer) — useful for hands-on tokenization experiments.

Communities & Further Learning

- Anthropic's official developer documentation and changelog for staying current on model updates.
- Provider release notes (Claude, GPT, Gemini) — model capabilities change frequently; always check current docs.